

Appendix: Symbols and Glossary

Daniel Ari Friedman
ORCID: 0000-0001-6232-9096
Email: daniel@activeinference.institute
DOI: 10.5281/zenodo.16887791

October 07, 2026

Contents

1 Appendix: Symbols and Glossary	1
1.1 Sets and Spaces	1
1.2 Quadray Coordinates and Geometry	1
1.3 Optimization and Algorithms	2
1.4 Information Theory and Geometry	2
1.5 Embeddings and Distances	3
1.6 Greek Letters (usage)	3
1.7 Notes (usage and cross-references)	3
1.8 Polyhedra and Synergetic Shapes	4
1.9 Acronyms and abbreviations	4
1.10 API Index (auto-generated; Methods linkage)	4

1 Appendix: Symbols and Glossary

This appendix consolidates the symbols, variables, and constants used throughout the manuscript.

1.1 Sets and Spaces

Symbol	Name
$\mathbb{R}^n$	Euclidean space
IVM	Isotropic Vector Matrix
Coxeter.4D	Euclidean 4D (E^4)
Einstein.4D	Minkowski spacetime (3+1)
Fuller.4D	Synergetics/Quadray tetrahedral space

Descriptions:

- $\mathbb{R}^n$: n -dimensional real vector space.
- IVM: Quadray integer lattice (CCP sphere centers).
- Coxeter.4D: Four-dimensional Euclidean geometry (not spacetime); see Coxeter, Regular Polytopes (Dover ed., p. 119); related lattice/packing background in Conway & Sloane.
- Einstein.4D: Relativistic spacetime with Minkowski metric.
- Fuller.4D: Quadrays with projective normalization and IVM unit conventions.

1.2 Quadray Coordinates and Geometry

Symbol	Name	Description
$q = (a, b, c, d)$	Quadray point	Non-negative coordinates with at least one zero after normalization
A, B, C, D	Quadray axes	Canonical tetrahedral axes mapped by the embedding
k	Normalization offset	$k = \min(a, b, c, d)$ used to set $q' = q - (k, k, k, k)$
q'	Normalized Quadray	Canonical representative with at least one zero and non-negative entries
$P_0, \dots, P_3$	Tetrahedron vertices	Vertices used in volume formulas
d_{ij}	Pairwise distances	Distance between vertices P_i and P_j (squared in CM matrix)
$\det(\cdot)$	Determinant	Determinant of a matrix
$ \cdot $	Magnitude	Absolute value (determinant magnitude)
V_{ivm}	Tetravolume (IVM)	Tetrahedron volume in synergetics/IVM units; unit regular tetra has $V_{ivm} = 1$
V_{xyz}	Tetravolume (XYZ)	Euclidean tetrahedron volume
$S3$	Scale factor	$S3 = \sqrt{9/8}$ with $V_{ivm} = S3 V_{xyz}$ (synergetics unit convention)
Coxeter.4D	Namespace	Euclidean E^4 ; regular polytopes
Einstein.4D	Namespace	Minkowski spacetime (metric analogy only here)
Fuller.4D	Namespace	Quadrays/IVM; integer tetravolume
Eq. (lattice_det)	Lattice determinant	Integer-lattice volume via 3x3 determinant
Eq. (ace5x5)	Tom Ace 5x5	Direct IVM tetravolume from Quadrays
Eq. (cayley_menger)	Cayley-Menger	Length-based formula: $288 V^2 = \det(\cdot)$

1.3 Optimization and Algorithms

Symbol	Name
α	Reflection coefficient
γ	Expansion coefficient
ρ	Contraction coefficient
σ	Shrink coefficient
V_{ivm}	Integer volume monitor

Descriptions:

- $\alpha, \gamma, \rho, \sigma$: Nelder-Mead parameters (typical values 1, 2, 0.5, 0.5).
- V_{ivm} : Tracks simplex volume across iterations.

1.4 Information Theory and Geometry

Symbol	Name	Description
$\log$	Natural logarithm	Logarithm base e
$\mathbb{E}[\cdot]$	Expectation	Mean with respect to a distribution
F_{ij}	Fisher Information Matrix	$\mathbb{E}[\partial_{\theta_i} \log p \cdot \partial_{\theta_j} \log p]$; Eq. (??) in the equations appendix
$\mathcal{F}$	Variational free energy	$-\log P(o \mid s) + \text{KL}[Q(s) \parallel P(s)]$; Eq. (??) in the equations appendix
$\text{KL}[Q \parallel P]$	Kullback-Leibler divergence	$\sum Q \log(Q/P)$; information distance
$\nabla_{\theta} L$	Natural gradient	$F(\theta)^{-1} \nabla_{\theta} L(\theta)$; Eq. (??) in the equations appendix
η	Step size	Learning-rate scalar used in updates
θ	Parameters	Model parameter vector; indices θ_i
ds^2	Minkowski line element	$-c^2 dt^2 + dx^2 + dy^2 + dz^2$; Eq. (??) in the equations appendix
c	Speed of light	Physical constant appearing in Minkowski metric

1.5 Embeddings and Distances

Symbol	Name	Description
M	Embedding matrix	Linear map from Quadray to $\mathbb{R}^3$ (Urner-style unless noted)
$\ \cdot\ _2$	Euclidean norm	$\sqrt{x_1^2 + \dots + x_n^2}$
R, D	Edge scales	Cube edge R and Quadray edge D with $D = 2R$ (common convention)

1.6 Greek Letters (usage)

Symbol	Name	Description
$\alpha, \gamma, \rho, \sigma$	NM coefficients	Nelder-Mead parameters (reflection, expansion, contraction, shrink)
θ	Theta	Parameter vector in models and metrics
μ	Mu	Internal states (Active Inference)
ψ	Psi	External states (Active Inference)
η	Eta	Step size / learning rate

1.7 Notes (usage and cross-references)

- **Figures referenced:** In-text references use LaTeX’s automatic figure numbering for consistent cross-referencing.
- **Equation references:** Use labels defined in the text (e.g., Eq. (??) in the equations appendix).

- **Namespaces:** We use Coxeter.4D, Einstein.4D, Fuller.4D consistently to designate Euclidean E^4 , Minkowski spacetime, and Quadray/IVM synergetics, respectively. This avoids conflation of Euclidean 4D objects (e.g., tesseracts) with spacetime constructs and synergetic tetravolume conventions.
- **External validation:** Cross-reference implementations from the [4dsolutions ecosystem](#) for algorithmic verification and performance comparison baselines. See the [Resources](#) section for comprehensive details.

1.8 Polyhedra and Synergetic Shapes

Symbol	Name	Description
Tetrahedron	Regular tetrahedron	Fundamental unit with $V=1$ in IVM units
Cube	Regular hexahedron	$V=3$ in IVM units; orthogonal space-filling
Octahedron	Regular octahedron	$V=4$ in IVM units; edge-midpoint construction
Rhombic Dodecahedron	12-faced solid	$V=6$ in IVM units; Voronoi cell of FCC packing
Cuboctahedron	Vector equilibrium	$V=20$ in IVM units; shell of 12 IVM neighbors
Truncated Octahedron	Archimedean solid	$V=20$ in IVM units; space-filling tiling

1.9 Acronyms and abbreviations

Acronym	Meaning
CM	Cayley-Menger (determinant-based tetrahedron volume)
PdF	Piero della Francesca (Heron-like tetrahedron volume)
GdJ	Gerald de Jong (Quadray-native tetravolume expression)
K-FAC	Kronecker-Factored Approximate Curvature (optimizer using structured Fisher)
CCP	Cubic Close Packing (same centers as FCC)
FCC	Face-Centered Cubic (same centers as CCP)
E^4	Four-dimensional Euclidean space (Coxeter.4D)
NM	Nelder-Mead (simplex optimization algorithm)
4dsolutions	Kirby Urner's GitHub organization with extensive Quadray implementations
BEAST	Synergetic modules (B, E, A, S, T) in Fuller's hierarchical system
OCN	Oregon Curriculum Network (educational framework integrating Quadrays)
POV-Ray	Persistence of Vision Raytracer (used in quadcraft.py visualizations)

1.10 API Index (auto-generated; Methods linkage)

The table below enumerates public symbols from `src/` modules.

Module	Symbol	Kind	Signature	Summary
quadmath.core.cayley_menger	ivm_tetra_volume_cayley_menger	function	(d2)	Compute IVM tetravolume from squared distances via Cayley-Menger.
quadmath.core.cayley_menger	squared_distances_from_quadray	function	(p0, p1, p2, p3, embedding)	Build the 4x4 squared-distance matrix from four quadray vertices.
quadmath.core.cayley_menger	tetra_circumradius	function	(d2)	Circumscribed sphere radius of a tetrahedron from squared distances.
quadmath.core.cayley_menger	tetra_inradius	function	(d2)	Inscribed sphere radius of a tetrahedron from squared distances.
quadmath.core.cayley_menger	tetra_volume_cayley_menger	function	(d2)	Compute Euclidean tetrahedron volume from squared distances (Coxeter.4D).
quadmath.core.examples	example_cuboctahedron_neighbors	function	()	Return twelve-around-one IVM neighbors (vector equilibrium shell).
quadmath.core.examples	example_cuboctahedron_vertices	function	()	Return XYZ coordinates for the twelve-around-one neighbors.
quadmath.core.examples	example_ivm_neighbors	function	()	Return the 12 nearest IVM neighbors as permutations of {2,1,1,0} (Fuller.4D).
quadmath.core.examples	example_optimize	function	()	Run Nelder-Mead over integer quadrays for a simple convex objective (Fuller.4D).
quadmath.core.examples	example_partition_tetra_volume	function	(mu, s, a, psi)	Construct a tetrahedron from the four-fold partition and return tetravolume (Fuller.4D).
quadmath.core.examples	example_volume	function	()	Return the exact IVM tetravolume of the primitive lattice tetrahedron.

Module	Symbol	Kind	Signature	Summary
quadmath.core.geometry	lorentz_factor	function	(v, c)	Lorentz factor $\gamma = 1 / \sqrt{1 - v^2/c^2}$ (Einstein.4D).
quadmath.core.geometry	minkowski_interval	function	(dt, dx, dy, dz, c)	Return the Minkowski interval squared ds^2 (Einstein.4D).
quadmath.core.geometry	proper_time	function	(dt, dx, dy, dz, c)	Proper time elapsed for a timelike interval (Einstein.4D).
quadmath.core.geometry	spacetime_classify	function	(ds2, tol)	Classify a Minkowski interval squared as timelike, spacelike, or lightlike.
quadmath.core.linalg_utils	bareiss_determinant_int	function	(matrix)	Compute an exact integer determinant using the Bareiss algorithm.
quadmath.core.linalg_utils	bareiss_rank	function	(matrix)	Compute the exact integer rank of a matrix via Bareiss elimination.
quadmath.core.linalg_utils	integer_adjugate	function	(matrix)	Compute the exact integer adjugate (classical adjoint) of a square matrix.
quadmath.core.metrics	angle_error	function	(q1, q2)	Geodesic rotation angle between two quaternions, in radians.
quadmath.core.metrics	fim_eigenspectrum	function	(F)	Eigen-decomposition of a Fisher information matrix.
quadmath.core.metrics	fisher_condition_number	function	(F)	Compute the condition number of the Fisher information matrix.
quadmath.core.metrics	fisher_curvature_analysis	function	(F)	Comprehensive analysis of Fisher information matrix curvature.
quadmath.core.metrics	fisher_quadray_comparison	function	(F_cartesian, F_quadray)	Compare Fisher information matrices between coordinate systems.
quadmath.core.metrics	fisher_rao_metric	function	(p, q, eps)	Fisher-Rao geodesic distance on the probability simplex.

Module	Symbol	Kind	Signature	Summary
quadmath.core.metrics	information_length	function	(path_gradients)	Gradient-weighted proxy for informational path length (NOT the Jensen-Shannon divergence between two discrete distributions.
quadmath.core.metrics	jensen_shannon_divergence	function	(p, q, eps)	Kullback-Leibler divergence between two discrete distributions.
quadmath.core.metrics	kl_divergence	function	(p, q, eps)	Root-mean-square chordal dispersion of quaternions about their mean.
quadmath.core.metrics	quat_log_euclidean_dispersion	function	(quats)	Shannon entropy $H(p)$ for a discrete distribution.
quadmath.core.metrics	shannon_entropy	function	(p, eps)	Quadray vector with non-negative components and at least one zero (Fuller.4D).
quadmath.core.quadray	DEFAULT_EMBEDDING	constant	`quadmath.core.quadray` `Quadray` class	Tom Ace 5x5 determinant as the exact IVM tetra-volume (Fuller.4D).
quadmath.core.quadray	ace_tetravolume_5x5	function	(p0, p1, p2, p3)	Angle at vertex q2 formed by rays q2->q1 and q2->q3 (radians).
quadmath.core.quadray	angle	function	(q1, q2, q3, embedding)	Component-wise mean of quadray points, rounded to the nearest lattice point.
quadmath.core.quadray	centroid	function	(*quads)	Euclidean distance between two quadray points under the given embedding.
quadmath.core.quadray	distance	function	(q1, q2, embedding)	Return Euclidean dot product $\langle q1, q2 \rangle$ under the given embedding.
quadmath.core.quadray	dot	function	(q1, q2, embedding)	Compute the exact IVM tetra-volume of a lattice tetrahedron (Fuller.4D).
quadmath.core.quadray	integer_tetra_volume	function	(p0, p1, p2, p3)	

Module	Symbol	Kind	Signature	Summary
quadmath.core.quadray	magnitude	function	(q, embedding)	Return the Euclidean magnitude of q under the given embedding (vector norm).
quadmath.core.quadray	qconjugate	function	(q)	Conjugate (w, -x, -y, -z) of a quaternion in (w, x, y, z) order.
quadmath.core.quadray	qmul	function	(a, b)	Hamilton product of two quaternions.
quadmath.core.quadray	qrotate	function	(q, v_xyz, angle)	Rotate a 3-vector by a unit quaternion via Rodrigues ($v' = q v q^*$).
quadmath.core.quadray	quadray_from_xyz	function	(x, y, z, embedding)	Map an R^3 point back to the quadray lattice via pseudoinverse rounding.
quadmath.core.quadray	rotate_about_axis	function	(v_xyz, axis_xyz, angle)	Rotate a 3-vector about an axis by an angle (axis-angle convenience).
quadmath.core.quadray	slerp	function	(qa, qb, t)	Shortest-arc spherical linear interpolation between unit quaternions.
quadmath.core.quadray	to_xyz	function	(q, embedding)	Map quadray to R^3 via a 3x4 embedding matrix (Fuller.4D -> Coxeter.4D slice).
quadmath.core.symbolic	cayley_menger_volume_symbolic	function	(d2)	Return symbolic Euclidean tetrahedron volume from squared distances.
quadmath.core.symbolic	convert_xyz_volume_to_ivm	function	(V_xyz)	Convert a symbolic Euclidean volume to IVM tetravolume via S3.
quadmath.inference.information	action_update	function	(action, free_energy_fn, step_size, epsilon)	Continuous-time action update: $da/dt = -dF/da$.
quadmath.inference.information	active_inference_step	function	(mu, action, free_energy_fn, derivative_operator, step_size, epsilon)	Joint perception-action update step in Active Inference.
quadmath.inference.information	expected_free_energy	function	(log_p_o_given_s, q, p, log_p_o)	Expected free energy for Active Inference with prior preferences.

Module	Symbol	Kind	Signature	Summary
quadmath.inference. information	finite_difference_gradien	function	(function, x, epsilon)	Compute numerical gradient of a scalar function via central differences.
quadmath.inference. information	fisher_information_matrix	function	(gradients, normalize)	Estimate the Fisher information matrix via sample gradients.
quadmath.inference. information	fisher_information_quadray	function	(gradients, embedding_matrix)	Compute Fisher information matrix in both Cartesian and Quadray coordinates.
quadmath.inference. information	free_energy	function	(log_p_o_given_s, q, p)	Variational free energy for discrete latent states.
quadmath.inference. information	information_gain	function	(prior, posterior, eps)	Information gain (Bayesian surprise) between prior and posterior.
quadmath.inference. information	information_geometric_distance	function	(F, x1, x2)	Compute information-geometric distance between two points.
quadmath.inference. information	mutual_information	function	(p_joint, eps)	Mutual information $I(X; Y)$ from a joint probability matrix.
quadmath.inference. information	natural_gradient_step	function	(gradient, fisher, step_size, ridge)	Compute a natural gradient step using a damped inverse Fisher.
quadmath.inference. information	perception_update	function	(mu, derivative_operator, free_energy_fn, step_size, epsilon)	Continuous-time perception update: $d\mu/dt = D \mu - dF/d\mu$.
quadmath.lattice. conversions	embedding_basis	function	(M)	Return the four embedding COLUMNS as a (4,3) basis matrix (Fuller.4D axes).
quadmath.lattice. conversions	quadray_roundtrip	function	(q, M)	Assert the exact round-trip identity $recon == q$ for $q \rightarrow XYZ \rightarrow quadray$.
quadmath.lattice. conversions	quadray_to_xyz	function	(q, M)	Map a Quadray to Cartesian XYZ via a 3x4 embedding matrix (Fuller.4D $\rightarrow$ Coxeter.4D slice).

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. conversions	urner_embedding	function	(scale)	Return a 3x4 Urner-style symmetric embedding matrix (Fuller.4D -> Coxeter.4D slice).
quadmath.lattice. conversions	xyz_to_quadray_canonical	function	(xyz, M)	Recover the canonical integer quadray for an XYZ point in the embedding image, EXACTLY.
quadmath.lattice. ivm_dynamics	DynamicsParams	class	Parameters of a discrete IVM lattice dynamics run. `quadmath.lattice.ivm_dynamics` `FitResult` class	Outcome of gradient-free coupling identification.
quadmath.lattice. ivm_dynamics	IVMLattice	class	Finite IVM lattice ball with neighbor adjacency and diffusion operators. `quadmath.lattice.ivm_dynamics` `Trajectory` class	Deterministic simulation record.
quadmath.lattice. ivm_dynamics	ball_sites	function	(radius)	Enumerate canonical quadray sites with squared IVM radius $\leq$ radius**2.
quadmath.lattice. ivm_dynamics	fit_trajectory	function	(observed, grid, lattice, kind, refine_rounds)	Identify the coupling alpha from an observed trajectory (gradient-free).
quadmath.lattice. ivm_dynamics	heat_step	function	(u, lattice, alpha)	One heat-diffusion step $u \leftarrow (1 - \alpha) u + \alpha S u$.
quadmath.lattice. ivm_dynamics	is_nonincreasing	function	(values, tol)	True iff no consecutive pair of values increases by more than tol.
quadmath.lattice. ivm_dynamics	majority_step	function	(u, lattice, alpha)	One rounded-averaging ("majority") step on integer states.
quadmath.lattice. ivm_dynamics	make_lattice	function	(radius)	Build the finite IVM lattice ball of the given radius.
quadmath.lattice. ivm_dynamics	neighbor_shifts	function	()	Return the 12 canonical IVM neighbor shifts as quadray deltas.

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. ivm_dynamics	render_dynamics_demo	function	(output_path, radius=, seed=, alpha=, horizon=, fit_alpha=, grid_count=, refine_rounds=)	Render the multi-snapshot IVM dynamics demo figure; return its path.
quadmath.lattice. ivm_dynamics	simulate	function	(T, params, lattice, u0)	Simulate T updates; deterministic given params (fixed seed).
quadmath.lattice. ivm_dynamics	site_radius_sq	function	(q)	Integer squared IVM radius of a quadray site (squared embedding norm).
quadmath.lattice. ivm_dynamics	step	function	(u, lattice, params)	Apply one update of the dynamics in params to the field.
quadmath.lattice. ivm_dynamics	sum_of_squares	function	(u)	Sum of squares of a field — the observable of the heat lemma.
quadmath.lattice. ivm_field	IVMField	class	Scalar field over an IVM lattice ball, stored on a deterministic site index. `quadmath. lattice.ivm_field` `IVM_NEIGHBOR_STEPS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. ivm_field	TetrahedronFit	class	Result of :func:` fit_geometry`. ` quadmath.lattice. ivm_field` ` fit_geometry` function `(points, labels, embedding)` Least-squares recovery of a tetrahedron's orientation+scale from noisy 3D points. `quadmath.lattice. ivm_field` ` is_ivm_site` function `(q)` Return True iff q (after normalization) is an IVM lattice site . `quadmath. lattice.ivm_field` ` quadrayshell_norm` function `(q)` Return the IVM shell norm of q : an even integer equal to 2k . `quadmath.lattice. ivm_field` ` shell_ball_sites` function `(radius)` Enumerate the IVM lattice ball of the given shell radius, deterministically. `quadmath.lattice. ivm_field` ` shell_cardinalities` function `(max_shell)` Cardinalities of shells 0 .. max_shell (cuboctahedral numbers) . `quadmath. lattice.ivm_field` ` shell_sites` function `(k)` Enumerate all IVM lattice sites with quadrayshell norm 2k . `quadmath.lattice. .lattice_search` ` nearest` function `(site, R, k)` Return the k nearest IVM lattice sites within radius R . ` quadmath.lattice. lattice_search` ` squared_distance`	

Module	Symbol	Kind	Signature	Summary
quadmath.lattice. omni_numbering	NEIGHBOR_MOVES	constant	<pre> `quadmath. lattice.omni_numbering ` `clear_shell_cache ` function `( )` Drop memoized shell enumerations so the next call recomputes them. `quadmath. lattice.omni_numbering ` `cumulative_count` function `(k)` Return the total number of IVM sites through shellk(inclusive). ` quadmath.lattice. omni_numbering` ` generate_shell` function `(k)` Generate the sites of shellkof the omnidirectional close packing. `quadmath .lattice. omni_numbering` ` shell_count` function `(k)` Return the number of IVM sites on shellkof the close packing. `quadmath.lattice. omni_numbering` ` site_at_index` function `(index, max_shell)` Return the IVM site at a canonical global index . `quadmath. lattice.omni_numbering ` `site_index` function `(site, max_shell)` Return the canonical global index of an IVM site, or -1 if absent. ` quadmath.lattice. omni_numbering` ` sites_through_shell` function `( max_shell)` Return all IVM sites through shellmax_shellin canonical order. ` quadmath.learn. learning_eval` ` CrossValidationResult` class </pre>	K-fold cross-validation table for the IVM field learner.

Module	Symbol	Kind	Signature	Summary
quadmath.learn. learning_eval	GradientDescentTrainer	class	Full-batch gradient-descent fit of a linear model on standardized features. `quadmath.learn.learning_eval` `LearningCurveResult` class	Data-coverage curve for the IVM field learner.
quadmath.learn. learning_eval	RidgeSiteFit	class	Closed-form ridge fit of a single linear site model. `quadmath.learn.learning_eval` `TrajectorySplitResult` class	Outcome of a temporal train/test evaluation of dynamics identification.
quadmath.learn. learning_eval	cross_validate_field	function	(field_values, sites, k, lam_grid, seed, radius=, kernel_width=)	Cross-validate the Laplacian-regularized field learner over k folds.
quadmath.learn. learning_eval	enclosing_radius	function	(sites)	Return the smallest IVM ball radius containing every given site.
quadmath.learn. learning_eval	kfold_site_splits	function	(observed_sites, k, seed)	Partition observed lattice sites into k seeded folds.
quadmath.learn. learning_eval	learning_curve	function	(values, sites, train_fracs, seed, radius=, lam=, kernel_width=)	Trace held-out MSE as a function of the fraction of observed sites.
quadmath.learn. learning_eval	ridge_site_fit	function	(features, values, lam)	Fit a closed-form ridge regression with intercept on tabular rows.
quadmath.learn. learning_eval	three_way_split	function	(n, train_frac, val_frac, seed)	Split range(n) into seeded, disjoint train/val/test index lists.
quadmath.learn. learning_eval	trajectory_train_test	function	(observed, split_frac, lattice, kind=, grid=, refine_rounds=)	Identify dynamics parameters on a training prefix, score the held-out suffix.

Module	Symbol	Kind	Signature	Summary
quadmath.optimize. discrete_variational	DiscretePath	class	Optimization trajectory on the integer quadray lattice. `quadmath .optimize. discrete_variational` `apply_move` function `(q, delta) ` Apply a lattice move and normalize to the canonical representative. ` quadmath.optimize. discrete_variational` ` discrete_ivm_descent` function ` objective, start, moves=, max_iter=, on_step=)` Greedy discrete descent over the quadray integer lattice. `quadmath .optimize. discrete_variational` `neighbor_moves_ivm` function `()`` Return the 12 canonical IVM neighbor moves as Quadray deltas. `quadmath. optimize. nelder_mead_quadray` `SimplexState` class (vertices, exclude_idx)	
quadmath.optimize. nelder_mead_quadray	centroid_excluding	function	(vertices, exclude_idx)	Integer centroid of three vertices, excluding the specified index.
quadmath.optimize. nelder_mead_quadray	compute_volume	function	(vertices)	Exact IVM tetra-volume (a Fraction, absolute determinant divided by 4) of the first four vertices.
quadmath.optimize. nelder_mead_quadray	nelder_mead_quadray	function	(f, initial_vertices, alpha, gamma, rho, sigma, max_iter, tol, on_step)	Nelder-Mead on the integer quadray lattice.
quadmath.optimize. nelder_mead_quadray	order_simplex	function	(vertices, f)	Sort vertices by objective value ascending and return paired lists.

Module	Symbol	Kind	Signature	Summary
quadmath.optimize. nelder_mead_quadray	project_to_lattice	function	(q)	Project a quadray to the canonical lattice representative via normalize.
quadmath.paths	get_data_dir	function	()	Return quadmath/output/data path and ensure it exists.
quadmath.paths	get_figure_dir	function	()	Return quadmath/output/figures path and ensure it exists.
quadmath.paths	get_output_dir	function	()	Return quadmath/output path at the repo root and ensure it exists.
quadmath.paths	get_repo_root	function	(start)	Heuristically find repository root by walking up from start.
quadmath.pipeline	FieldLearner	class	Laplacian-regularized field learner over an IVM ball; satisfies :class:`Fittable`. `quadmath.pipeline` `FieldModel` class	Anything that predicts a scalar field value at a lattice site.
quadmath.pipeline	Fittable	class	Anything that can fit a field model from data, then predict. `quadmath.pipeline` `LatticeBall` class	Immutable view of an IVM lattice ball; satisfies :class:LatticeSource.
quadmath.pipeline	LatticeSource	class	Anything that can enumerate IVM lattice sites (structural). `quadmath.pipeline` `Pipeline` class	Immutable sequence of :class:Step objects; monoid-style composition.

Module	Symbol	Kind	Signature	Summary
quadmath.pipeline	Step	class	A named, typed callable unit of a : class:`Pipeline`. `quadmath.pipeline` `dynamics_step` function `(params, T )` Step simulating the discrete IVM dynamics in params (ignores input). ` quadmath.pipeline` ` learn_step` function `(lam, seed, radius )` Step fitting an : class:`IVMField` by Laplacian-regularized learning. ` quadmath.pipeline` ` sites_step` function `(radius)` Step producing the IVM ball sites of shell radius (ignores input) . `quadmath.stats. benchmarks` ` BENCH_DEFAULTS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.stats. benchmarks	BenchRow	class	One timed benchmark result. `quadmath. stats.benchmarks` ` bench_conversions` ` function   `(n, trials)`   Benchmark quadray /XYZ conversions over Ndeterministic samples.     `quadmath .stats.benchmarks`   ` bench_field_fit`   ` function `(n_sites, trials)` Benchmark IVMField.learnon a synthetic field with a fixed seed. ` quadmath.stats. benchmarks` ` bench_lattice_search` function `(n_sites , queries, trials)` Benchmark nearest-site queries through the lattice_searchball index. `quadmath. stats.benchmarks` ` bench_shell_enumeration ` function `(k_max , trials)` Benchmark shell enumeration through shellk_max. `quadmath.stats. benchmarks` `run_all ` function `()`` Run every benchmark with the module-level :data:`BENCH_DEFAULTS `. `quadmath.stats .benchmarks` ` summary_table` function `(rows)` Render aligned fixed- width ASCII rows as a table. `quadmath. stats.benchmarks` ` time_callable` function `(fn, trials=, warmup=)` Timefnwith time.perf_counterand return per-trial wall seconds. ` quadmath.stats. statistics` ` benjamini_hochberg` function `(pvals)` Benjamini-Hochberg FDR-adjusted p-values (step-up procedure).	

Module	Symbol	Kind	Signature	Summary
quadmath.tools. glossary_gen	build_api_index	function	(src_dir)	
quadmath.tools. glossary_gen	generate_markdown_table	function	(entries)	
quadmath.tools. glossary_gen	inject_between_markers	function	(markdown_text, begin, end, payload)	
quadmath.validate. validate	DEFAULT_CHECKS	constant	`quadmath. validate.validate` ` DEFAULT_TOLERANCE` constant	
quadmath.validate. validate	NOTES	constant	`quadmath. validate.validate` ` SKIPPED_CHECKS` constant	

Module	Symbol	Kind	Signature	Summary
quadmath.validate. validate	ValidationReport	class	Immutable outcome of a single validation check. `quadmath. validate.validate` ` check_associativity` ` function   `(a, b, c, tol)`   Verify $(a*b)*$ $c == a*(b*c)$ within <b>tol</b> via the core Hamilton product.     `quadmath.validate. validate`   ` check_conjugate_inverse `   function   `(q, tol)`   Verify $q *$ $\text{conj}(q)$ equals the identity $(1, 0, 0, 0)$ within <b>tol</b> .     ` quadmath.validate. validate`   ` check_double_cover`   function   `(q1, q2, tol)`   Verify the $S^0$ $(3)$ homomorphism $R(q1*$ $q2) == R(q1) R(q2)$ within <b>tol</b> .     ` quadmath.validate. validate`   ` check_normalization`   function   `(q, tol)`   Verify the quaternion norm is within <b>tol</b> of 1 (norm of $(a, b, c, d)$ ).     ` quadmath.validate. validate`   ` check_slerp_midpoint`   function   `(q0, q1, tol)`   Verify the shortest-arc slerp midpoint lies on the geodesic of $(q0, q1)$ .     `quadmath.validate .validate`   ` run_validation`   function   ` quaternions, checks)`   Run checks over <b>quaternions</b> and collect deterministic reports.     `quadmath .viz._common`   ` atomic_target`   function   `(path)`   Yield a temporary sibling of <b>path</b> ; move it onto <b>path</b> only on success.     `quadmath .viz._common`   ` 	A single animation frame.

Module	Symbol	Kind	Signature	Summary
quadmath.viz. animations	GRID_SIZE	constant	`quadmath.viz. animations` ` diffusion_frames` function `(n_steps, seed)` Render explicit heat diffusion on the IVM radius-3 ball adjacency. ` quadmath.viz. animations` ` frames_strip` function `(frames, out_path, labels, save )` Render frames as a single-row matplotlib strip, one panel per frame. ` quadmath.viz. animations` ` frames_to_gif` function `(frames, out_path, fps, scale)` Assemble frames into an animated GIF at <u>out_path</u> . ` quadmath.viz. animations` ` lattice_frames` function `(shells, n )` Render a pulsing IVM lattice ball. ` `quadmath.viz. animations`   ` simplex_frames`   function   `(q0, q1, n)`   Render a quaternion-slerp rotation of the IVM radius-1 ball.     ` quadmath.viz.plots`   `plot_error_histogram`   function   `(errors , bins, save, out_path)`   Plot a histogram of error values with a dashed vertical mean line.     `quadmath. viz.plots`   ` plot_lattice_shell_3d`   function   `(k, save, out_path)`   Scatter the sites of one IVM lattice shell in 3D (equal-aspect axes).     `quadmath. viz.plots`   ` plot_loss_history`   function   `(losses,	

Module	Symbol	Kind	Signature	Summary
quadmath.viz. vis_lattice	GALLERY_FILES	constant	`quadmath.viz. vis_lattice` ` dynamics_strip` ` function   `(axs, trajectory, t_indices, embedding=, cmap=, titles=)`   Render evolution snapshots of a trajectory as a strip of 3D panels.     `quadmath.viz. vis_lattice`   ` field_slice`   ` function `(ax, field , sites, plane, q0=, cmap=, title=, colorbar=)` Heatmap of a scalar IVM field restricted to a lattice plane. ` quadmath.viz. vis_lattice` ` gallery` function `(paths_out_dir, seed) ` Compose the three lattice-gallery figures deterministically. `quadmath.viz. vis_lattice` ` shell_scatter` ` function   `(ax, sites , k, embedding=, color =, size=, axis_hints=, title=)`   Scatter one IVM frequency shell in 3D with tetrahedral axis hints .     `quadmath.viz. vis_stats`   ` GALLERY_FILES` constant (paths_out_dir, seed)	
quadmath.viz.vis_stats	gallery	function		Compose the four statistics-gallery figures deterministically.
quadmath.viz.vis_stats	plot_ciBars	function	(labels, means, lows, highs, ax, title=)	Point estimates with confidence intervals [lows, highs] as error bars.

Module	Symbol	Kind	Signature	Summary
quadmath.viz.vis_stats	plot_ecdf	function	(values, ax, title=)	Empirical cumulative distribution function as a sorted step plot.
quadmath.viz.vis_stats	plot_latency_hist	function	(times, ax, bins=, title=)	Histogram of a latency sample with a dashed vertical mean line.
quadmath.viz.vis_stats	plot_scaling_loglog	function	(sizes, times, ax, title=)	Log-log scatter of times versus sizes with the fitted power law.
quadmath.viz.visualize	animate_discrete_path	function	(path, embedding, save, out_path)	Animate a point moving along a discrete quadray path.
quadmath.viz.visualize	animate_simplex	function	(vertices_list, embedding, save, out_path)	Animate simplex evolution across iterations.
quadmath.viz.visualize	plot_ivm_neighbors	function	(embedding, save, out_path)	Scatter the 12 IVM neighbor points in 3D.
quadmath.viz.visualize	plot_partition_tetrahedron	function	(mu, s, a, psi, embedding, save, out_path)	Plot the four-fold partition as a labeled tetrahedron in 3D.
quadmath.viz.visualize	plot_simplex_trace	function	(state, save, out_path)	Plot per-iteration diagnostics for Nelder-Mead.